

1. PROBLEM STATEMENT / QUESTION

Assignment Objective:

Design and implement a modular banking application in Java demonstrating the core pillars of Object-Oriented Programming (OOP): *Data Encapsulation*, *Inheritance*, and *Runtime Polymorphism (Method Overriding)*.

- Base Class Account:** Encapsulate account credentials (accountnumber, accountholder) as private members and maintain a balance field. Provide accessor methods and core transaction facilities: deposit(), withdraw(), calinterest(), transfer(), and show().
- Subclass SavingsAccount:** Inherit from Account and introduce interest management via interestrate. Override withdraw() to restrict overdrafts and enforce non-negative balances. Override calinterest() to credit computed percentage interest.
- Subclass CurrentAccount:** Inherit from Account and implement business overdraft capabilities via overdraftlimit. Override withdraw() to permit withdrawals exceeding current balance up to the overdraft threshold. Override calinterest() to assess maintenance service fees.
- Driver Execution:** In class LabProgram1, instantiate account objects, simulate deposits, withdrawals, interest evaluations, and inter-account fund transfers, printing the state of accounts before and after transactions.

1. Encapsulation

Restricts direct access to sensitive fields (accountnumber, accountholder) using private access specifiers and exposes controlled read access via public getters.

2. Inheritance

Promotes code reusability: SavingsAccount and CurrentAccount extend the parent Account class, inheriting base attributes and constructor mechanics using super().

3. Polymorphism

Dynamic method dispatch allows subclasses to override withdraw() and calinterest(), delivering specialized banking rules tailored to each account type at runtime.

2. STEP-BY-STEP PROCESS

Stage	Phase	Technical Implementation Details
Step 1	Base Class Construction	Declare class Account. Define private String accountnumber, private String accountholder, and public double balance. Initialize them using a parameterized constructor using this.
Step 2	Accessor & Core Methods	Implement getters (getaccountnumber(), getaccountholder(), getbalance()). Implement deposit() with validation against non-positive inputs, empty method templates for withdraw() & calinterest(), and utility show().
Step 3	Inter-Account Transfers	Implement transfer(Account fixedAccount, double amount) within the base class, reusing this.withdraw(amount) followed by fixedAccount.deposit(amount).
Step 4	Savings Account Specialization	Create SavingsAccount extends Account. Add field interestrate. Override withdraw() with validation: check if amount > 0 and amount <

Stage	Phase	Technical Implementation Details
		balance. Override <code>calinterest()</code> to compute and credit percentage interest.
Step 5	Current Account Specialization	Create <code>CurrentAccount</code> extends <code>Account</code> . Add field <code>overdraftlimit</code> . Override <code>withdraw()</code> to permit deductions up to <code>balance + overdraftlimit</code> . Override <code>calinterest()</code> to levy maintenance charges.
Step 6	Driver Class & Testing	In <code>LabProgram1.main()</code> , instantiate a <code>CurrentAccount</code> ("UB 24311") and a <code>SavingsAccount</code> ("sb 24533"). Execute financial transactions sequentially and print intermediate and final states.

3. ALGORITHM

1. Initialization:

- Create `CurrentAccount ca = new CurrentAccount("UB 24311", "kumar", 4500)`.
- Create `SavingsAccount sa = new SavingsAccount("sb 24533", "heamnth", 5000)`.

2. **Initial State Display:** Invoke `ca.show()` and `sa.show()` to output initial balances.

3. Current Account Operations:

- `ca.deposit(2000)`: Validate `amount > 0`; update `balance = 4500 + 2000 = 6500.0`.
- `ca.withdraw(2500)`: Validate `amount ≤ (balance + overdraftlimit)`; update `balance = 6500 - 2500 = 4000.0`.
- `ca.calinterest()`: Apply flat service fee; update `balance = 4000 + 50.0 = 4050.0`.

4. Savings Account Operations:

- `sa.withdraw(3000)`: Validate `amount < balance`; update `balance = 5000 - 3000 = 2000.0`.
- `sa.calinterest()`: Compute `interest = 2000.0 * 0.0 = 0.0`; update `balance = 2000.0 + 0.0 = 2000.0`.

5. Fund Transfer:

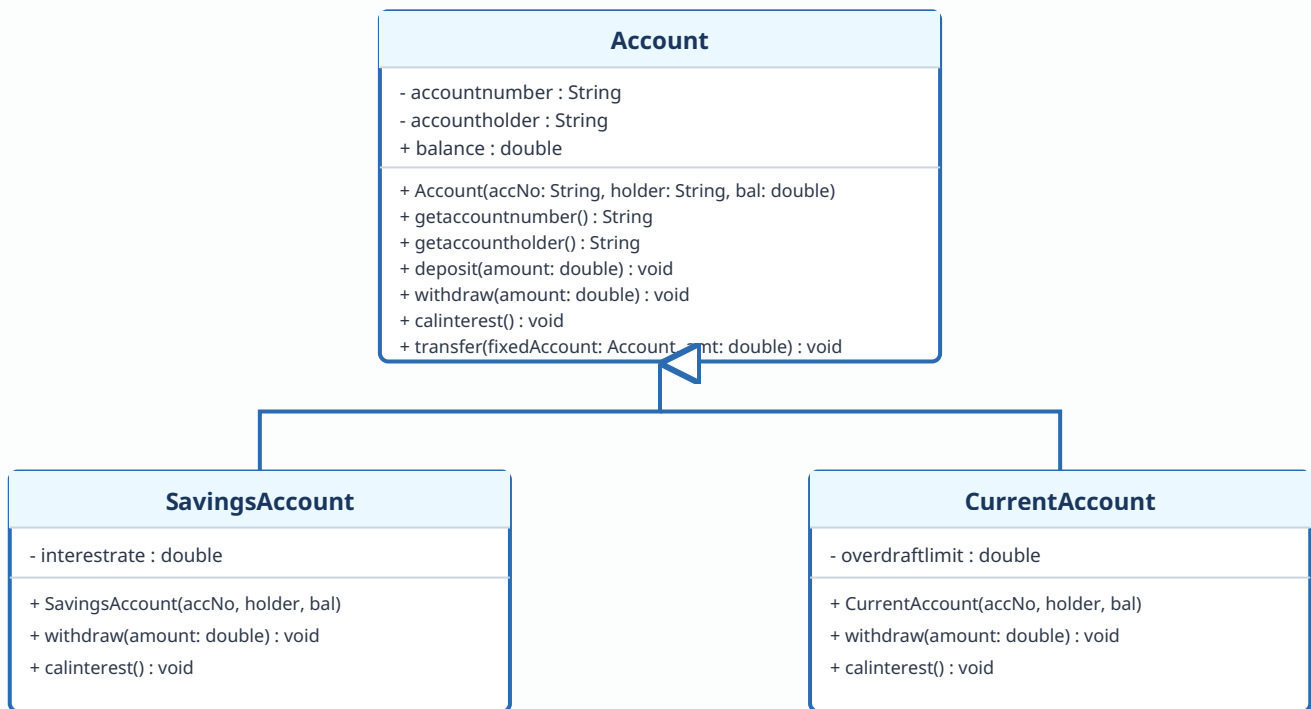
- Execute `sa.transfer(ca, 1000)`:
 - Invoke `sa.withdraw(1000)`: `balance = 2000 - 1000 = 1000.0`.
 - Invoke `ca.deposit(1000)`: `balance = 4050 + 1000 = 5050.0`.

6. **Final State Display:** Invoke `ca.show()` and `sa.show()` to print updated balances.

7. End Program.

4. UML CLASS DIAGRAM

The class diagram illustrates the class relationships, field visibility (- private, + public), inheritance hierarchies (generalization arrows), and method signatures.



5. JAVA SOURCE CODE

```

class Account {
    private String accountnumber; // details are hidden (encapsulation)
    private String accountholder;
    public double balance;

    public Account(String accountnumber, String accountholder, double balance) {
        this.accountnumber = accountnumber; // constructor parameter initialization
        this.accountholder = accountholder;
        this.balance = balance;
    }

    public String getaccountnumber() { // getter for accountnumber
        return accountnumber;
    }

    public String getaccountholder() { // getter for accountholder
        return accountholder;
    }

    public double getbalance() {
        return balance;
    }

    void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("deposit amount is greater than zero");
            return;
        }
        balance += amount;
        System.out.println(" the deposited amount is " + amount);
    }
}
  
```

```

    void withdraw(double amount) {}

    void calinterest() {}

    void transfer(Account fixedAccount, double amount) {
        System.out.println("transfer of " + amount + " from " + accountnumber + " to " + fixedAccount.getAccountnumber());
        this.withdraw(amount);
        fixedAccount.deposit(amount);
    }

    void show() {
        System.out.println("ACCOUNT NUMBER IS " + accountnumber);
        System.out.println("ACCOUNT HOLDER IS " + accountholder);
        System.out.println("BALANCE IS " + balance);
    }
}

class SavingsAccount extends Account {
    private double interestrate;

    public SavingsAccount(String accountnumber, String accountholder, double balance) {
        super(accountnumber, accountholder, balance);
        this.interestrate = interestrate; // default initialized or passed rate
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("with draw amount is not possible ");
        } else if (amount >= balance) {
            System.out.println("it has insufficient balance");
        } else {
            balance -= amount;
            System.out.println("withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double interest = balance * interestrate;
        balance += interest;
        System.out.println("interest is credited" + interest);
    }
}

class CurrentAccount extends Account {
    private double overdraftlimit;

    public CurrentAccount(String accountnumber, String accountholder, double balance) {
        super(accountnumber, accountholder, balance);
        this.overdraftlimit = overdraftlimit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("with draw amount is not possible ");
        } else if (amount > balance + overdraftlimit) {
            System.out.println("it has insufficient balance and the limited is done");
        } else {
            balance -= amount;
            System.out.println("withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double charge = 50.0;
        balance += charge;
        System.out.println("maintainence charge is credited" + charge);
    }
}

```

```

    }
}

public class LabProgram1 {
    public static void main(String[] args) {
        CurrentAccount ca = new CurrentAccount("UB 24311", "kumar", 4500);
        SavingsAccount sa = new SavingsAccount("sb 24533", "heamnth", 5000);

        System.out.println(" initial account details");
        ca.show();
        System.out.println("");
        sa.show();

        System.out.println("transaction details");
        ca.deposit(2000);
        ca.withdraw(2500);
        ca.calinterest();

        sa.withdraw(3000);
        sa.calinterest();
        sa.transfer(ca, 1000);

        System.out.println("final account details");
        ca.show();
        System.out.println("");
        sa.show();
    }
}

```

6. SAMPLE EXECUTION OUTPUT

```

    initial account details
ACCOUNT NUMBER IS UB 24311
ACCOUNT HOLDER IS kumar
BALANCE IS 4500.0

ACCOUNT NUMBER IS sb 24533
ACCOUNT HOLDER IS heamnth
BALANCE IS 5000.0

transaction details
    the deposited amount is 2000.0
withdraw is 2500.0
maintenance charge is credited50.0
withdraw is 3000.0
interest is credited0.0
transfer of 1000.0 from sb 24533 to UB 24311---
withdraw is 1000.0
    the deposited amount is 1000.0

final account details
ACCOUNT NUMBER IS UB 24311
ACCOUNT HOLDER IS kumar
BALANCE IS 5050.0

ACCOUNT NUMBER IS sb 24533
ACCOUNT HOLDER IS heamnth
BALANCE IS 1000.0

```